Android UI Layouts

Android Layout

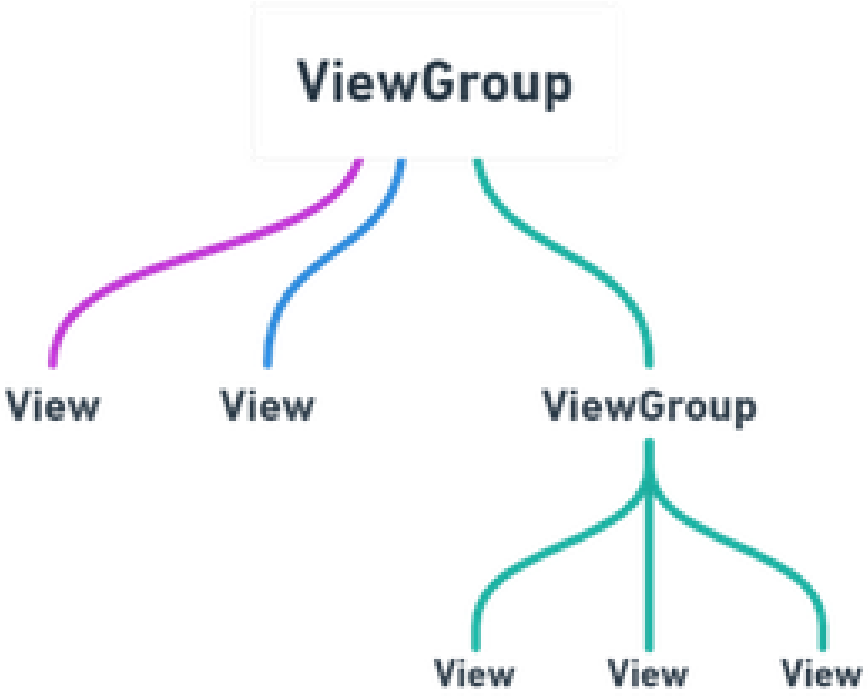
- Layout Managers (or simply layouts) are said to be extensions of the **ViewGroup** class.
- They are used to **set the position of child Views** within the UI we are building.
- We can nest the layouts, and therefore we can create arbitrarily complex UIs using a combination of layouts.
- There is a **number of layout classes** in the Android SDK.
- They can be used, modified or can create your own to make the UI for your Views, Fragments and Activities.
- You can display your contents effectively by using the right combination of layouts.

View

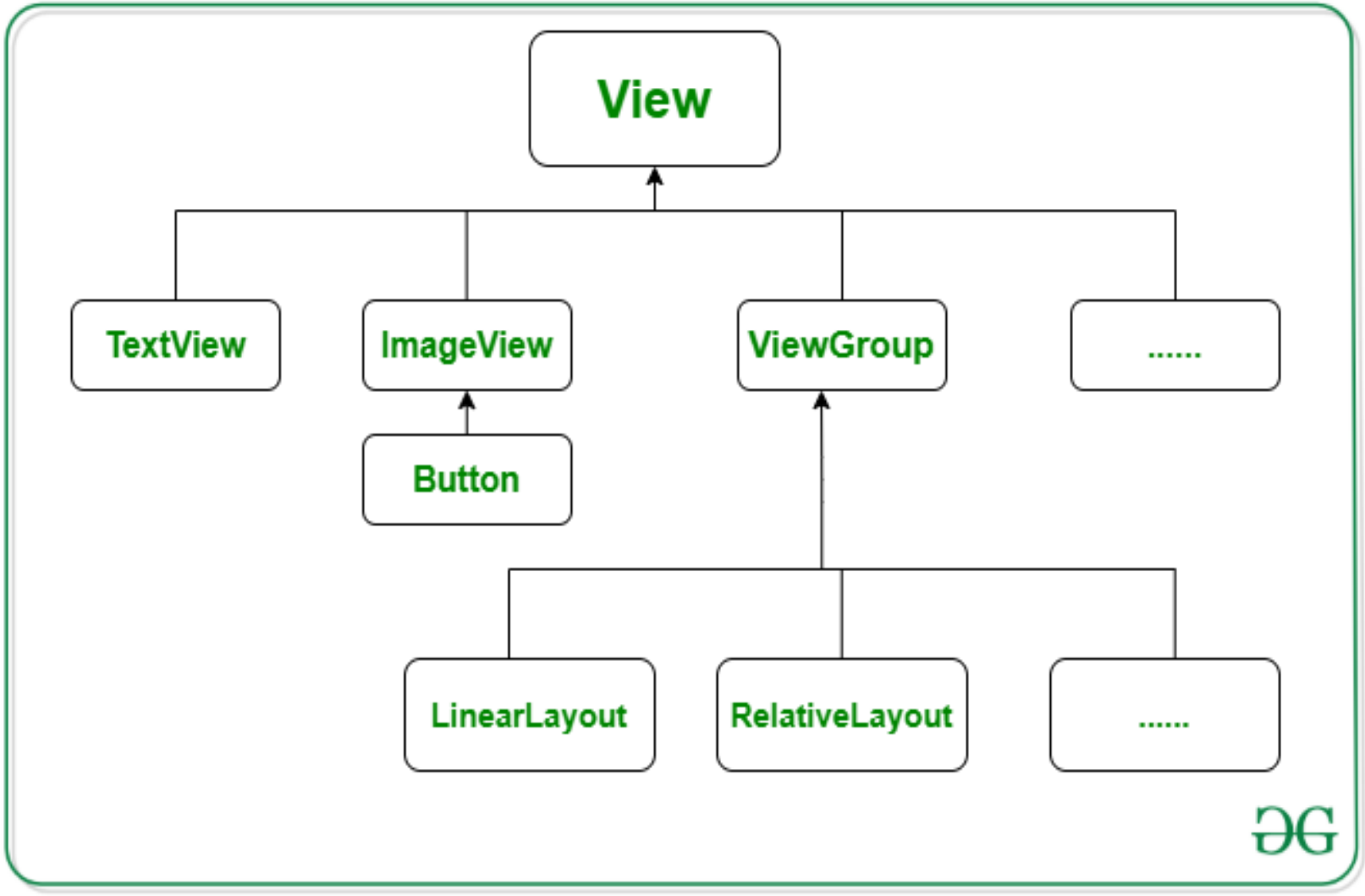
- The **basic building block for user interface** is a View object which is created from the View class and occupies a rectangular area on the screen and is **responsible for drawing and event handling**.
- A **View** is defined as the user interface which is used to create interactive UI components such as **TextView, ImageView, EditText, RadioButton**, etc., and is responsible for event handling and drawing.
- They are Generally Called Widgets.

ViewGroup

- A **ViewGroup** act as a base class for layouts and layouts parameters that hold other Views or ViewGroups and to define the layout properties. They are Generally Called layouts.



ViewGroup


View
TextView
ImageView
ViewGroup
.....
Button
LinearLayout
RelativeLayout
.....

Types of Android Layout

- **Android Linear Layout**
- **Android Relative Layout**
- **Android Absolute Layout**
- **Android Frame Layout**
- **Android Table Layout**
- **Android ListView**
- **Android Grid Layout**

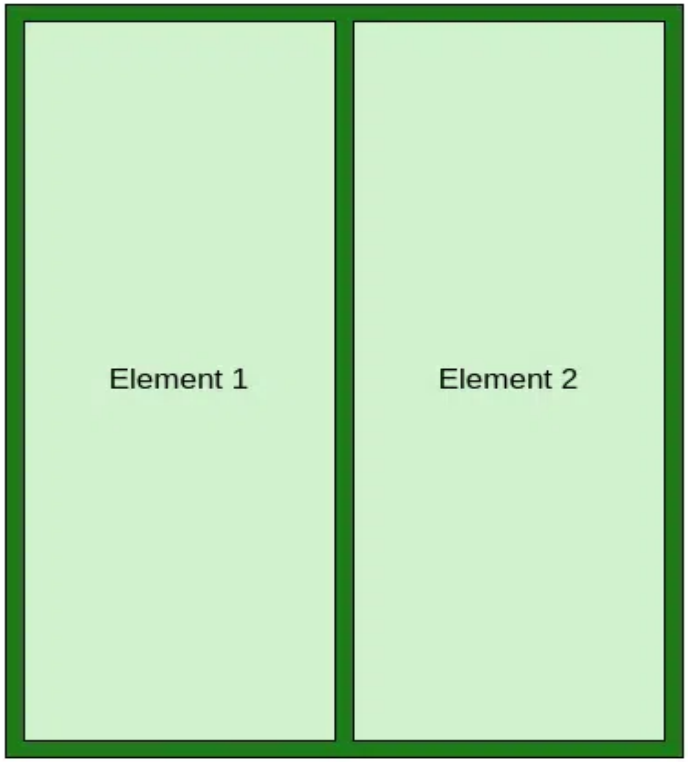
Different Attribute of the Layouts

XML attributes	Description
android:id	Used to specify the id of the view.
android:layout_width	Used to declare the width of View and ViewGroup elements in the layout.
android:layout_height	Used to declare the height of View and ViewGroup elements in the layout.
android:layout_marginLeft	Used to declare the extra space used on the left side of View and ViewGroup elements.
android:layout_marginRight	Used to declare the extra space used on the right side of View and ViewGroup elements.
android:layout_marginTop	Used to declare the extra space used in the top side of View and ViewGroup elements.
android:layout_marginBottom	Used to declare the extra space used in the bottom side of View and ViewGroup elements.
android:layout_gravity	Used to define how child Views are positioned in the layout.

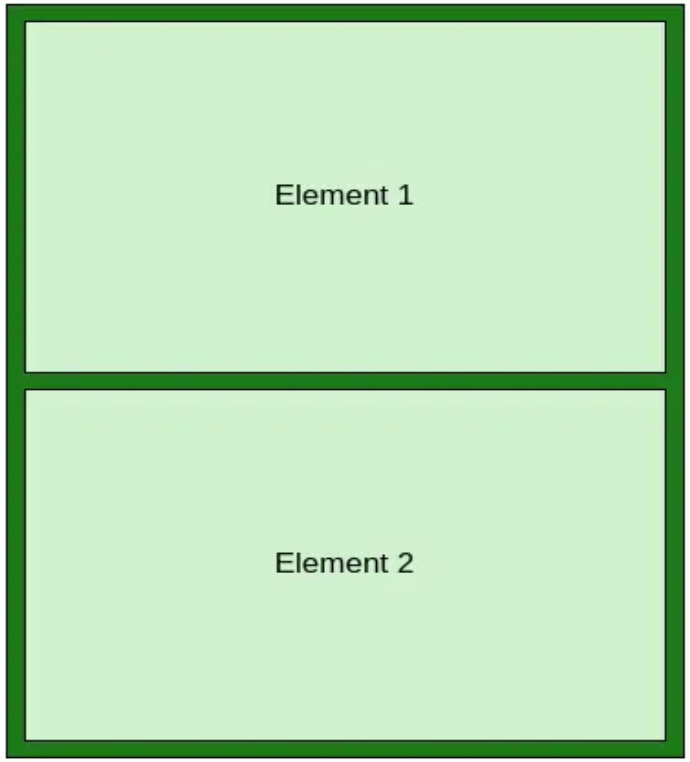
Linear Layout

- LinearLayout in Android is a ViewGroup subclass, used to **arrange child view elements one by one in a singular direction** either **horizontally or vertically** based on the orientation attribute.
- We can specify the linear layout orientation using the **android:orientation** attribute.
- All the child elements arranged one by one in either multiple rows or multiple columns.
    - Horizontal list: One row, multiple columns.
    - Vertical list: One column, multiple rows.

Linear Layout



Horizontal Linear Layout

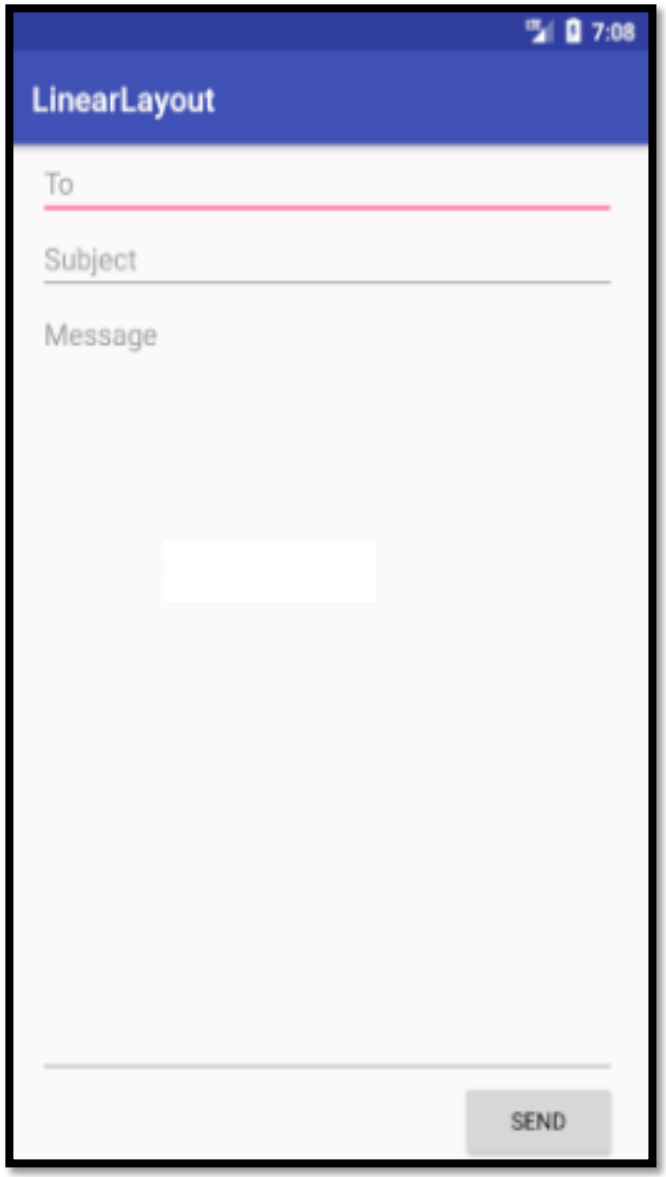


Vertical Linear Layout

```
<?xml version="1.0" encoding="utf-8"?>
    <LinearLayout xmlns:android="http://schem
    as.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingLeft="20dp"
        android:paddingRight="20dp"
        android:orientation="vertical" >
        <EditText
            android:id="@+id/txtTo"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="To"/>
        <EditText
            android:id="@+id/txtSub"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Subject"/>
```

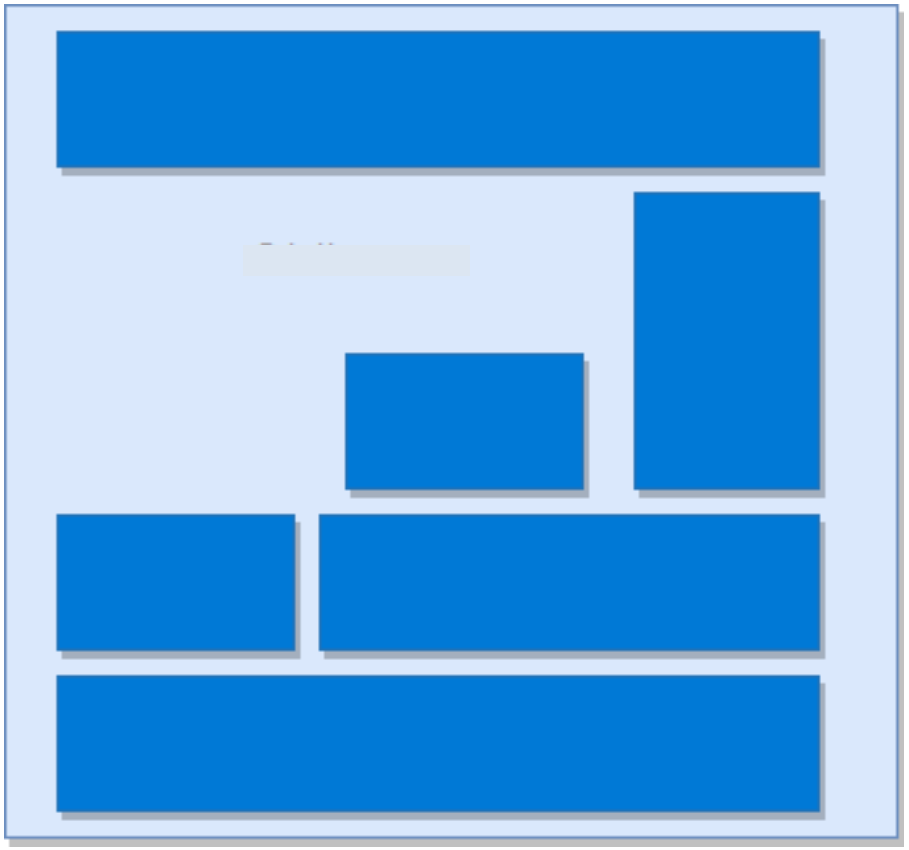
```
  <EditText
        android:id="@+id/txtMsg"
      android:layout_width="match_parent“
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="top"
        android:hint="Message"/>
<Button
    android:layout_width="100dp"
    android:layout_height="wrap_content"
        android:layout_gravity="right"
        android:text="Send"/>
</LinearLayout>
```

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



Relative Layout

- It is flexible than other native layouts as it lets us to define the position of **each child View relative to the other views** and the dimensions of the screen.



XML attributes	Description
layout_alignParentLeft	Set “true” to match the left edge of view to the left edge of parent.
layout_alignParentRight	Set “true” to match the right edge of view to the right edge of the parent.
layout_alignParentTop	Set to “true” to match the top edge of the view to the top edge of the parent.
layout_alignParentBottom	Set to “true” to match the bottom edge of the view to the bottom edge of the parent.
layout_alignLeft	It accepts another sibling view ID and Align the view to the left of the specified view ID.
layout_alignRight	It accepts another sibling view ID and Align the view to the right of the specified view ID.
layout_alignStart	It accepts another sibling view ID and Align the view to the start of the specified view ID.
layout_alignEnd	It accepts another sibling view ID and Align the view to the end of the specified view ID.

XML attributes	Description
layout_centerInParent	When it is set to “true”, the view will be aligned to the center of parent.
layout_centerHorizontal	When it is set to “true”, the view will be horizontally centre-aligned within its parent.
layout_centerVertical	When it is set to “true”, the view will be vertically centre-aligned within its parent.
layout_toLeftOf	It accepts another sibling view ID and places the view left of the specified view ID.
layout_toRightOf	It accepts another sibling view ID and places the view right of the specified view ID.
layout_toStartOf	It accepts another sibling view ID and places the view to start of the specified view ID.
layout_toEndOf	It accepts another sibling view ID and places the view to end of the specified view ID.
layout_above	It accepts another sibling view ID and places the view above the specified view ID.
layout_below	It accepts another sibling view ID and places the view below the specified view ID.

```
<?xml version="1.0" encoding="utf-8"?>
    <RelativeLayout xmlns:android="http://sche
    mas.android.com/apk/res/android"
       android:layout_width="match_parent"
       android:layout_height="match_parent"
       android:paddingLeft="10dp"
       android:paddingRight="10dp">
       <Button
          android:id="@+id/btn1"
          android:layout_width="wrap_content"
          android:layout_height="wrap_content"
          android:layout_alignParentLeft="true"
          android:text="Button1" />
       <Button
          android:id="@+id/btn2"
          android:layout_width="wrap_content"
          android:layout_height="wrap_content"
          android:layout_alignParentRight="true"
          android:layout_centerVertical="true"
          android:text="Button2" />
```

```
<Button
    android:id="@+id/btn3"
 android:layout_width="wrap_content"
android:layout_height="wrap_content"
 android:layout_alignParentLeft="true"
    android:layout_centerVertical="true"
    android:text="Button3" />

<Button
    android:id="@+id/btn4"
 android:layout_width="match_parent"
android:layout_height="wrap_content
android:layout_alignParentBottom="true"
    android:text="Button4" />
```

```
<Button
        android:id="@+id/btn5"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
      android:layout_alignBottom="@+id/btn2"
       android:layout_centerHorizontal="true"
        android:text="Button5" />
     <Button
        android:id="@+id/btn6"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/btn4"
      android:layout_centerHorizontal="true"
        android:text="Button6" />
```

```
  <Button
      android:id="@+id/btn7"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
     android:layout_toEndOf="@+id/btn1"
    android:layout_toRightOf="@+id/btn1"
    android:layout_alignParentRight="true"
      android:text="Button7" />
</RelativeLayout>
```

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {


   @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
       setContentView(R.layout.activity_main);
    }
}
```

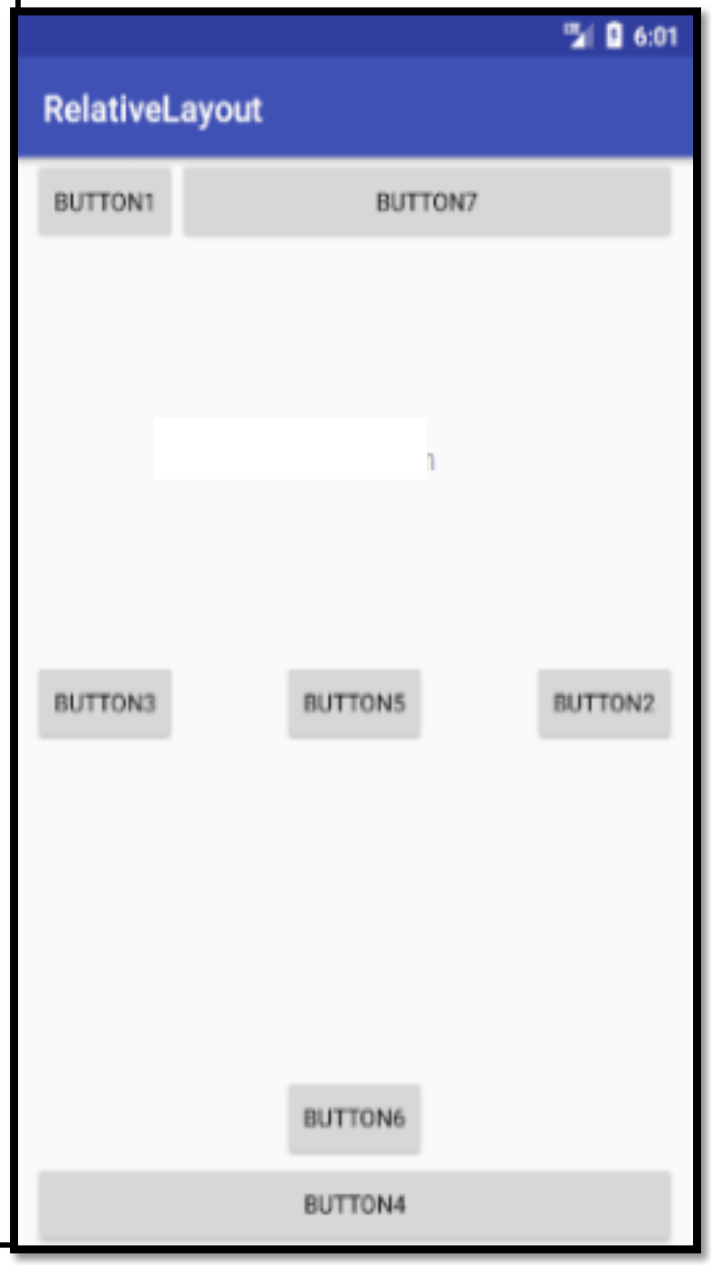


Table Layout

- Table Layout will position its children elements into rows and columns and it won't display any border lines for rows, columns or cells.

- The Table Layout in android will work same as the HTML table and the table will have as many columns as the row with the most cells.

- The TableLayout can be explained as **<table>** and **TableRow** is like **<tr>** element.

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

        <TableRow
          android:layout_width="fill_parent"
          android:layout_height="fill_parent">
              <TextView
                    android:text="Time"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_column="1" />

                <TextClock
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:id="@+id/textClock"
                    android:layout_column="2" />
        </TableRow>
```

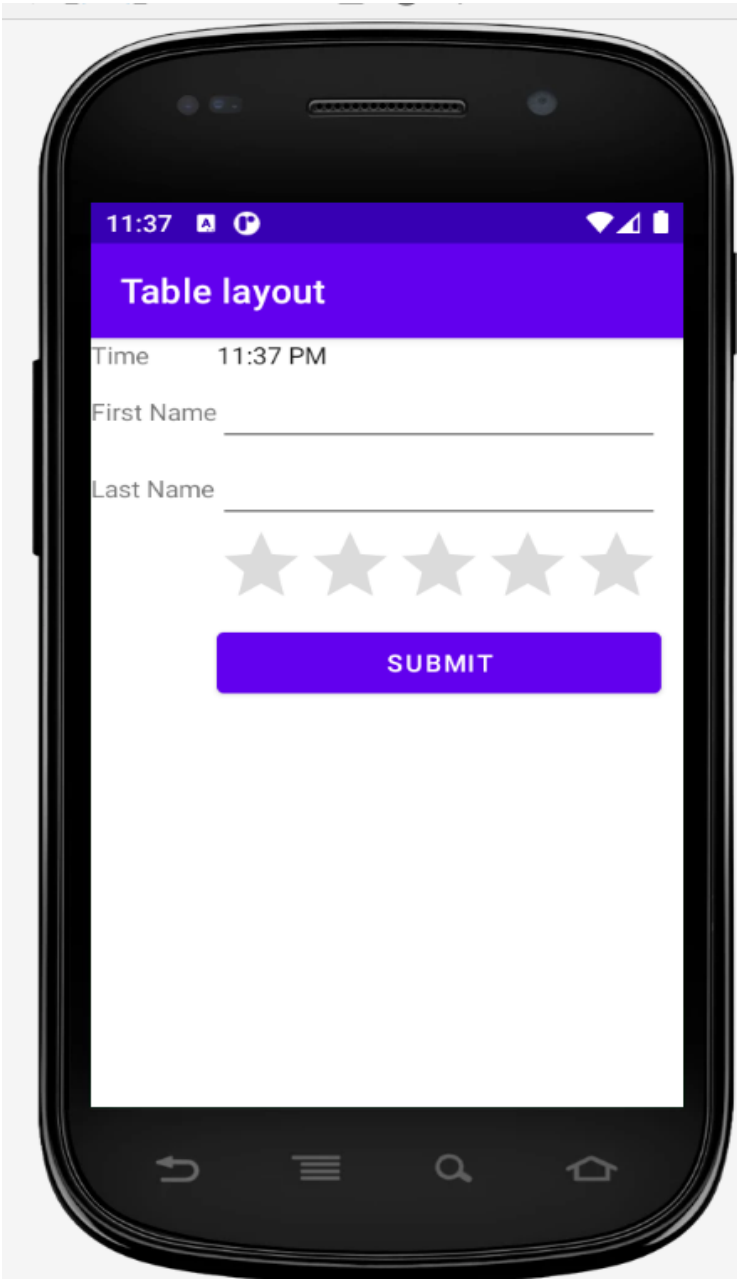
```
<TableRow
        android:layout_width="fill_parent"
        android:layout_height="fill_parent">

         <TextView
            android:text="First Name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_column="1" />

        <EditText
            android:width="200px"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />

</TableRow>
```

```
<TableRow>

        <TextView
            android:text="Last Name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_column="1" />

        <EditText
            android:width="100px"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />

</TableRow>
```

```
<TableRow
        android:layout_width="fill_parent"
        android:layout_height="fill_parent">

        <RatingBar
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/ratingBar"
            android:layout_column="2" />

</TableRow>
```

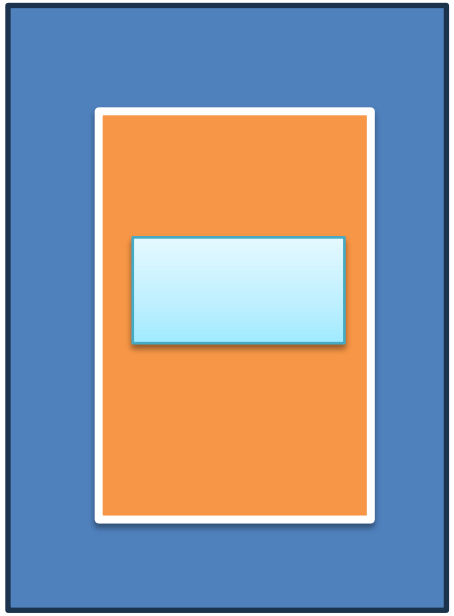
```
<TableRow
        android:layout_width="fill_parent"
        android:layout_height="fill_parent">

        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Submit"
            android:id="@+id/button"
            android:layout_column="2" />

    </TableRow>

    </TableLayout>
```

```
package com.example.tablelayout;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



Frame Layout

- Android Framelayout is a ViewGroup subclass that is used to specify the position of **multiple views placed on top of each other** to represent a single view screen.
- Generally, we can say FrameLayout simply blocks a particular area on the screen to display a single view.
- Here, all the child views or elements are added in stack format means the **most recently added child will be shown on the top of the screen**.
- But, we can add multiple children's views and control **their positions only by using gravity attributes** in FrameLayout.



```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

      <ImageView
       android:src="@drawable/logo"
       android:scaleType="fitCenter"
       android:layout_height="fill_parent"
       android:layout_width="fill_parent"/>

      <TextView
       android:text="Frame Demo"
       android:textSize="30px"
       android:textStyle="bold"
       android:layout_height="fill_parent"
       android:layout_width="fill_parent"
       android:gravity="center"/>

</FrameLayout>
```

Logo image can be added as follows –
Go to Resource Manager→
Click on + →
Import Drawables →
Select logo file →
ok

```
package com.example.myapplication1;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```